



第13章：代码生成与代码大模型



学习目标与主线

01

明确代码生成的核心任务范畴与特殊性，掌握代码大模型的定义、典型代表及核心应用场景（如自然语言转代码、代码补全 / 翻译 / 重构）。

02

理解代码生成的范式演变逻辑，梳理从早期符号方法、统计模型到现代大模型（尤其是 GPT 系列衍生模型）的技术迭代脉络与关键突破点。

03

掌握代码大模型的核心技术方法（预训练、微调与提示、辅助工具），以及其面临的正确性、可靠性等挑战与对应的评估指标（如 pass@k）。



引言与全景

代码生成任务的核心定义



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

📄 自然语言生成代码

将人类的自然语言需求描述，自动转化为可执行的计算机程序代码。

✍️ 代码自动补全

在程序员编写代码的过程中，实时预测并提供后续代码片段的智能建议。

↔️ 代码翻译与重构

实现跨编程语言的代码转换，或对现有代码进行结构优化与性能提升。

⚠️ 代码生成的特殊性：远超NLP的挑战

🛡️ 语法严格 Strict Syntax

代码是 **形式化语言**，任何语法错误都会导致程序无法编译或运行。模型必须生成 **完全符合语法规范** 的代码，容不得半点差错。

💡 对比：自然语言即使有语法错误，仍能理解大意

🎯 语义精确 Semantic Accuracy

生成的代码不仅要语法正确，更要 **准确实现用户意图**。逻辑错误同样会导致程序失效，这要求模型具备 **深层的逻辑推理能力**。

💡 关键转折：2022年思维链提示(CoT)让模型学会“一步步想问题”

⚠️ 核心动因阐释

🛡️ 代码的强形式化属性

代码语法严格、语义精确且容错率趋近于零，需专门训练掌握语法规则、API 用法及代码逻辑，与自然语言需求差异显著。

🛡️ 任务场景的特殊性

涵盖自然语言转代码、跨语言翻译等专属任务，需针对性学习“需求 - 代码”映射及语言差异，通用模型无法覆盖。

🛡️ 专业知识的强依赖性

需融合算法、数据结构等专业知识，需在海量开源代码中学习代码模式与工程规范，才能精准落地用户意图。

代码大模型：概念与行业代表



核心定义

代码大模型是指专门在 **海量开源代码库**（如GitHub）上进行预训练或微调的 **大型语言模型**。它们被优化以理解和生成代码，从而显著提升代码相关任务的表现。



OpenAI Codex

GitHub Copilot核心模型

HumanEval pass@1 28.8%

对比GPT-3 0% → 28.8%

✓ 基于GPT架构，在 **179GB Python代码** 上微调



DeepMind AlphaCode

编程竞赛级表现

竞赛排名 前54.3%

解决问题数 10/10

✓ 在Codeforces竞赛中达到 **人类选手水平**



BigCode StarCoder

开源模型领导者

支持语言 80+

HumanEval 领先开源

✓ 由Hugging Face等推出，注重 **开放与协作**



CodeLlama

Meta开源力作

HumanEval pass@1 53.7%

参数量 7B/13B/34B

✓ 基于LLaMA 2，在 **500B+代码token** 上训练

💡 **关键趋势：** 闭源模型性能领先，但开源模型快速追赶，技术民主化加速

Open + Closed

技术演进：从符号方法到现代大模型



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

1960s-2000s 早期符号方法

⚙️ 基于人工规则

开发者手动编写大量语法规则和逻辑模板，本质是代码片段的“拼图游戏”。

✔️ 核心特点

可控与可预测：生成结果完全由预设规则决定，行为固定，调试简单。

❌ 局限性

灵活性差：难以处理复杂多变需求，无法泛化，维护成本极高。

2000s-2018 统计语言模型

📊 数据驱动学习

模型从大量代码数据中学习词法和句法统计规律，建立概率生成机制。

↑ 生成连贯性提升

能生成更连贯代码片段，初步实现简单代码补全功能，提升编码效率。

⚠️ 模型容量局限

受限于模型结构，难以捕捉长距离依赖和深层复杂语义逻辑。

2018-Present 现代大模型：范式革命

🧠 强理解能力

深度理解自然语言复杂意图，精准转化为可执行代码。

✂️ 端到端生成

基于Codex，通过海量代码微调，实现高质量端到端生成。

🔧 编程范式革新

开发者从“编写”转向“描述”功能，借助Copilot等工具极大提升效率。

📈 性能飞跃数据

86.6%
GPT-4 pass@1

92.1%
Claude 3.5

80.9%
Claude Opus SWE

30%+
效率提升

🔑 革命性转变：从“告诉计算机如何做”到“告诉计算机做什么”

How → What



核心架构设计

核心技术方法：预训练、微调与提示工程

预训练 Pre-training

海量代码语料

在数以TB计的开源代码库（如GitHub）上进行自监督学习

多维学习目标

掌握编程语言语法结构、常见模式、API使用方法及深层语义关联

关键优化技巧

- ✓ 文本-代码混合
- ✓ 跨语言预训练
- ✓ 保留代码注释

微调 Fine-tuning

有监督微调 (SFT)

使用高质量的“自然语言描述-代码”配对数据进行训练

适配特定任务

使模型更擅长根据指令生成代码，是适应具体任务的关键步骤

性能提升显著

Codex在179GB Python代码上微调，pass@1从0%提升至28.8%

提示工程 Prompt Engineering

Few-shot提示

提供少量示例引导模型模仿生成，显著提升输出质量

指令微调

基于大量多样指令微调，增强对自然语言指令的理解和执行能力

思维链 (CoT)

2022年突破，让模型学会“一步步想问题”，推理能力大幅提升

训练数据规模对比

179GB

Codex Python代码
5400万个仓库

500B+

CodeLlama token
多语言混合

3TB

StarCoder数据
80+编程语言

2.6T

Baichuan 2 token
通用+代码混合

性能对比：主流代码大模型评估数据

👑 闭源模型（性能领先）

Claude Opus 4.1 SOTA
HumanEval: 92.1% SWE-bench: 70.3%

GPT-5 高效
HumanEval: 89.3% 成本节省: 90%

Gemini 2.5 Pro 长上下文
HumanEval: 87.6% 上下文: 2M tokens

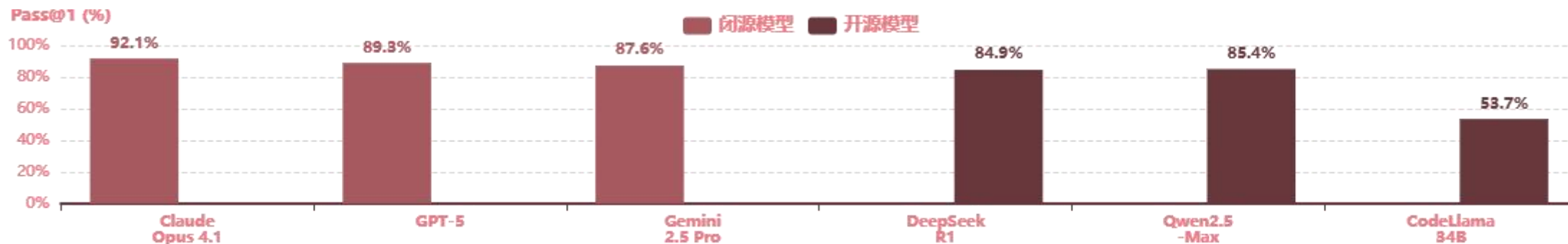
🔓 开源模型（快速追赶）

DeepSeek R1 开源最强
HumanEval: 84.9% 特点: MoE架构

Qwen2.5-Max 国产领先
HumanEval: 85.4% 上下文: 128K

CodeLlama 34B Meta
HumanEval: 53.7% MBPP: 56.2%

📊 HumanEval Pass@1 性能对比图





能力扩展与评估

评估体系：HumanEval与pass@k指标

HumanEval数据集

数据集构成

164个手写编程问题，专注于衡量从文档字符串合成程序的**功能正确性**

每个问题包含

- ✓ 明确的函数签名
- ✓ 详尽的文档字符串
- ✓ 完整的函数体
- ✓ 平均**7.7**个单元测试

评估能力

语言理解、算法实现、简单数学推理等编程核心能力

pass@k指标详解

指标定义

衡量模型生成 k 个代码样本时，**至少有一个**通过测试的概率

计算公式

$$\text{pass@k} = E[1 - C(n-c, k) / C(n, k)]$$

n : 生成样本总数 | c : 正确样本数 | k : 评估样本数

评估流程

1. 为每个问题生成 n 个样本（通常 $n=200$ ）
2. 统计通过测试的正确样本数 c
3. 计算无偏估计 pass@k
4. 取所有问题的平均值

其他重要评估基准

MBPP

Mostly Basic Python Programming，包含约1000个Python编程问题

- ✓ 入门级编程任务

SWE-bench

使用真实GitHub问题，评估模型解决实际软件工程挑战的能力

- ✓ 真实世界场景

DS-1000

数据科学场景，涵盖7个Python库（NumPy、Pandas等）

- ✓ 数据科学专用



挑战、前沿与总结

四大核心挑战

正确性 Correctness

⚠ 语法错误

复杂代码中仍可能出现拼写错误或格式不规范，导致编译失败

✖ 逻辑错误

更隐蔽且致命，代码能运行但无法实现预期功能，需测试用例发现

安全扫描通过率

仅55%代码通过安全扫描（Veracode研究）

55%

可靠性 Reliability

⚡ 输出不稳定

模型输出对输入提示的微小变化非常敏感，导致生成结果不稳定

⚖ 逻辑不一致

难以保证生成代码逻辑的一致性，相同需求可能产生不同实现

📈 性能波动

不同运行环境下性能表现差异明显，影响实际应用效果

安全性 Security

⚠ 生成漏洞代码

可能生成包含 OWASP十大安全漏洞 的高风险代码

🔑 泄露敏感信息

存在泄露训练数据中硬编码密码或API密钥的风险

Java代码安全通过率

Java表现尤为突出，远低于其他语言

29%

泛化能力 Generalization

✓ 分布内任务

在训练数据频繁出现的**常见编程任务**上表现出色，能快速生成标准代码

✗ 分布外任务

面对 **新颖、少见** 或需要创造性思路的问题，往往束手无策

🔍 根本原因

模型本质上学习数据中的统计规律，而非真正理解编程逻辑，缺乏抽象推理能力

挑战：正确性

✖ 提升代码生成成功率的辅助工具



单元测试反馈

自动运行预设测试用例，模型根据失败信息迭代修正，形成闭环验证

✓ 错误定位 → 自动修复



语法检查与编译反馈

利用静态分析工具检查语法错误，将编译结果实时反馈给模型进行精准修复

✓ 静态分析 → 实时修复



交互式编程协作

在IDE中实现实时交互，根据上下文动态补全，构建高效的"人机协作"模式

✓ 实时交互 → 动态补全

🔑 未来发展方向



多模态能力

结合图像生成前端代码，实现从设计稿到代码的自动转换

→ UI设计 → 代码生成



长上下文处理

支持百万级token上下文，处理大型代码库和复杂项目

→ 2M → 10M+ tokens



安全性增强

自动漏洞检测与修复，提升生成代码的安全性和可靠性

→ 55% → 90%+ 安全率



领域专用优化

针对特定编程语言和框架的深度优化，提升专业领域表现

→ 通用 → 垂直领域

2024-2025

多模态突破

2025-2026

长上下文普及

2026+

安全性标准化

01

核心定位与任务范畴：代码生成是大模型应用中具有特殊属性的任务，涵盖自然语言转代码、代码补全 / 翻译 / 重构等子任务，其对语法严格性和语义精确性的要求远超自然语言处理，而代码大模型是经海量源代码预训练 / 微调的专用模型（如 Codex、AlphaCode 等），专为提升此类任务表现而生。

02

技术演进与核心方法：代码生成范式历经从早期符号方法、统计模型（n-gram）到现代大模型的跨越式发展，GPT 系列衍生模型（如 Codex）推动自然语言驱动编程成为现实；核心技术围绕预训练（文本 - 代码混合、跨语言等）、微调与提示（有监督微调、few-shot 提示、指令微调）展开，且需结合单元测试、语法检查等辅助工具提升生成

03

关键挑战与评估标准：代码生成面临正确性、可靠性、安全性、泛化能力四大核心挑战，生成代码需满足编译通过、功能达标等硬性要求；其评估不依赖常规自然语言指标（如 BLEU），而是以功能正确性为核心，pass@k 指标（k 次尝试中获得正确解的概率）成为衡量模型实际编程能力的关键标准。